

1. สรุปภาพรวมสถาปัตยกรรมและผลการอัปเดตระบบล่าสุด (System Architecture & Updates)

โครงสร้างองค์กร 4 Tiers

- Tier 1: ผู้บริหารคณะ (คณบดี/รองฯ)
- Tier 2: หัวหน้าภาควิชา & หัวหน้างาน
- Tier 3: อาจารย์ประจำภาควิชา
- Tier 4: บุคลากรสายสนับสนุน

Zero-Trust RBAC & Security

- กอด Quick Switcher: ตรวจสอบสิทธิ์จริง
- 2FA OTP: 6 หลักส่งเข้าอีเมลจริง (120s)
- High-Entropy Passwords: สุ่มรหัส
- First-Login: บังคับเปลี่ยนรหัสผ่าน

PostgreSQL 16 & Resilient

- สลับจาก SQLite ไป PostgreSQL 16
- Prisma ORM + ซิงค์ Supabase Cloud
- มีระบบ In-Memory Fallback กรณีรันบน Serverless (Vercel) ไร้ Downtime

DevOps & CI/CD Pipeline

- Multi-stage Dockerfile: Node 20 Non-root
- Docker Compose: Postgres + Web-app
- CI/CD GitHub Actions: ตรวจสอบ Prisma, Build Next.js & Docker Dry-run

2. สรุปเส้นทาง API ทั้งหมดในระบบ (21 Endpoints Master Catalog)

Method	Endpoint Path	หน้าที่การทำงานและพฤติกรรมของระบบ	Auth	สิทธิ์การเข้าถึง
POST	/api/auth/login	ตรวจสอบ ID & Password (รองรับ Employee ID/Email) และตั้งคูกี้ <code>pending_user_id</code> รอรับ OTP	Public	ทุกคน
POST	/api/auth/otp/send	ส่ง OTP 6 หลักแบบ Secure บันทึกลงตาราง <code>Otp</code> (หมดอายุ 120s) และยิงอีเมลจริงผ่าน SMTP	Pending	ผู้ใช้ที่ผ่าน Login
POST	/api/auth/otp/verify	ตรวจสอบรหัส OTP 6 หลัก เทียบกับ DB ปรับ <code>isUsed=true</code> และออก Session Cookie (TTL 8 ชม.)	Pending	ผู้ใช้ที่ผ่าน Login
POST	/api/auth/password/change	บังคับเปลี่ยนรหัสผ่านสำหรับการใช้งานครั้งแรก (ความยาว >= 6 ตัวอักษร) ปรับ <code>isFirstLogin=false</code>	Session	ล็อกอินครั้งแรก
POST	/api/auth/password/forgot	ส่งรหัส OTP 6 หลัก เพื่อกู้คืนรหัสผ่านเข้าอีเมลจริงของผู้ใช้งาน	Public	ทุกคนที่มี Email
POST	/api/auth/password/reset	ตั้งรหัสผ่านใหม่หลังจากผู้ใช้ยืนยัน OTP รีเซตรหัสผ่านสำเร็จ	Public	ผู้ยืนยัน OTP แล้ว
GET	/api/auth/me	ดึงข้อมูลโปรไฟล์ สังกัด และระดับ Tier ของผู้ใช้ปัจจุบันผ่าน Session Cookie	Session	ผู้ที่ล็อกอินแล้ว
POST	/api/auth/logout	ออกจากระบบและลบ Session Cookies (<code>session_user_id</code> , <code>pending_user_id</code>) ทั้งหมด	Session	ผู้ที่ล็อกอินแล้ว
GET	/api/users	ดึงรายชื่อบุคลากรทั้งหมดในระบบ แยกตามสังกัด ฝ่าย และตำแหน่ง	Session	ADMIN Only
POST	/api/users	สร้างบัญชีพนักงานใหม่ สุ่ม Employee ID (EMP-XXXX) + รหัสผ่านแกระง ซิงค์ DB และส่งอีเมลแจ้ง	Session	ADMIN Only
GET	/api/documents	ดึงเอกสารเรียงตาม Scope: FACULTY, DEPARTMENT, HIERARCHICAL, INDIVIDUAL	Session	ตามสิทธิ์ผู้รับ
POST	/api/documents	สร้างเอกสารเรื่องใหม่ กำหนด Priority (NORMAL, URGENT, SECRET) แนบไฟล์และระบุผู้รับ	Session	บุคลากรทุกระดับ
GET	/api/documents/[id]	ดูรายละเอียดเอกสารฉบับเต็ม พร้อม Auto Mark as Read บันทึก Timestamp ลงตาราง <code>ReadLog</code> ทันที	Session	ตามสิทธิ์ผู้รับ
PUT	/api/documents/[id]	แก้ไขรายละเอียดเอกสาร พร้อมปรับสถานะ <code>isEdited=true</code> แสดงป้าย (แก้ไขแล้ว) อัตโนมัติ	Session	ผู้สร้าง / ADMIN
DELETE	/api/documents/[id]	ลบเอกสารเรื่องนออกจากระบบ (Cascading delete ไปยัง ReadLogs และ Replies ที่เกี่ยวข้อง)	Session	ผู้สร้าง / ADMIN
GET	/api/documents/[id]/replies	ดึงประวัติการตอบกลับ บันทึกข้อความความเห็น และคำขอลา/รายงานผลของเอกสาร	Session	ผู้มีสิทธิ์ดูเอกสาร
POST	/api/documents/[id]/replies	ส่งข้อความตอบกลับ หรือยื่นคำขอลา (<code>isLeaveRequest: true</code>) พร้อมแนบไฟล์รายงานการปฏิบัติงาน	Session	ผู้มีสิทธิ์ดูเอกสาร
GET	/api/admin/stats	สถิติภาพรวมระบบ (จำนวนผู้ใช้, เอกสาร, ReadLogs, Replies) และสถานะการเชื่อมต่อ PostgreSQL	Public/Dev	ผู้ดูแลระบบ
GET	/api/admin/routes-list	แสดงรายการ API Routes ทั้งหมดในระบบแบบ Dynamic Reflection สำหรับการตรวจสอบหลังบ้าน	Public/Dev	ผู้ดูแลระบบ
POST	/api/seed	นำเข้าชุดข้อมูลทดสอบตัวอย่าง (5 Users, 3 Documents, ReadLogs, Replies) เข้าสู่ PostgreSQL	Dev Only	นักพัฒนา / Admin
GET	/api/seed/supabase	ซิงค์ชุดข้อมูลบุคลากรตัวอย่าง 12 Personas อ้างอิง FLAS KPS KU ขึ้นสู่ Supabase Cloud Database	Admin/Key	ผู้ดูแลระบบคลาวด์

3. สรุปคำสั่งติดตั้งระบบ & นโยบาย Git (Deployment & Branching Policy)

คำสั่งรันระบบผ่าน Docker Compose

```
cp .env.example .env          # 1. ตั้งค่า Environment Variables
docker compose up -d --build    # 2. Build & รัน PostgreSQL 16 + Next.js App
# Web Portal: http://localhost:3000 | PostgreSQL: localhost:5432 (DB: E-office)
```

นโยบายการรวมโค้ด (Git Policy)

- กิ่งพัฒนาหลัก: โค้ดทั้งหมดอยู่บนกิ่ง `develop` (ห้าม Push ขึ้น `main` เด็ดขาด)
- CI/CD Gate: การรวมโค้ดเข้า `main` ต้องสร้าง Pull Request และผ่าน GitHub Actions ครบทั้ง 2 Stages

เอกสารฉบับนี้จัดทำขึ้นสำหรับโครงการพัฒนาระบบ FLAS KPS KU E-OFFICE + คณะศิลปศาสตร์และวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน